

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  entity alu is
4      port (reg1, reg0:                in std_logic_vector(3 downto 0);
5            enA, enSL, enSR, enM, enN: in std_logic;
6            aluOut:                   out std_logic_vector(3 downto 0));
7  end;
8
9  architecture synth of alu is
10     signal a, b, c, d, e, carry:    std_logic_vector(3 downto 0);
11     begin
12         carry(3) <= ((reg1(3) and reg0(3)) or (reg1(3) and carry(2)) or
13                     (reg0(3) and carry(2))) and enA;
14
15         a(3) <=      (reg1(3) xor reg0(3) xor carry(2)) and enA;
16
17         carry(2) <= ((reg1(2) and reg0(2)) or (reg1(2) and carry(1)) or
18                     (reg0(2) and carry(1))) and enA;
19
20         a(2) <=      (reg1(2) xor reg0(2) xor carry(1)) and enA;
21
22         carry(1) <= ((reg1(1) and reg0(1)) or (reg1(1) and carry(0)) or
23                     (reg0(1) and carry(0))) and enA;
24
25         a(1) <=      (reg1(1) xor reg0(1) xor carry(0)) and enA;
26
27         carry(0) <= (reg1(0) and reg0(0)) and enA;
28
29         a(0) <=      (reg1(0) xor reg0(0)) and enA;
30
31         b(3) <= reg1(2) and enSL;
32         b(2) <= reg1(1) and enSL;
33         b(1) <= reg1(0) and enSL;
34         b(0) <= '0';
35
36         c(3) <= '0';
37         c(2) <= reg1(3) and enSR;
38         c(1) <= reg1(2) and enSR;
39         c(0) <= reg1(1) and enSR;
40
41         d(3) <= reg1(3) and enM;
42         d(2) <= reg1(2) and enM;
43         d(1) <= reg1(1) and enM;
44         d(0) <= reg1(0) and enM;
45
46         e(3) <= (not reg1(3)) and enN;
47         e(2) <= (not reg1(2)) and enN;
48         e(1) <= (not reg1(1)) and enN;
49         e(0) <= (not reg1(0)) and enN;
50
51         aluOut <= a or b or c or d or e;
52     end;
```